

Linux RAID 5 Lab Note: Create, break and repair a Redundant Storage Array

I completed a hands-on lab simulating enterprise storage resilience using Linux. When my local Chromebook environment restricted the required low-level system operations, I pivoted to **Google Cloud (GCP)** to spin up a secure, flexible sandbox environment.

Here is a breakdown of how I built, intentionally broke, and successfully repaired a RAID 5 array using `mdadm` and loop devices.

Part 1: Preparing the Virtual Disks

To simulate real hardware without buying physical drives, I created “fake” hard drives out of files using loop devices:

1. Create 3 empty disks: `fallocate -l 100M disk1.img disk2.img disk3.img`
2. Attach Disk 1 to the system: `sudo losetup -fP disk1.img`
3. Attach Disk 2 to the system: `sudo losetup -fP disk2.img`
4. Attach Disk 3 to the system: `sudo losetup -fP disk3.img` [here letter 'p' should be capital]
5. Confirm all their names (usually loop0,1,2): `losetup -a`

Part 2: Building the RAID5 Array

With the virtual disks ready, I combined them into a fault-tolerant RAID 5 configuration and mounted the filesystem:

6. Install the RAID tool: `sudo apt update && sudo apt install mdadm -y`
7. Create the RAID 5 Array: `sudo mdadm --create --verbose /dev/md0 --level=5 --raid-devices=3 /dev/loop0 /dev/loop1 /dev/loop2`
8. Now format the Array: `sudo mkfs.ext4 /dev/md0`
9. Next, create a folder to hold the files: `sudo mkdir -p /mnt/raid_lab`
10. Then, Plug in the RAID to that folder: `sudo mount /dev/md0 /mnt/raid_lab`

Part 3: Testing and Resilience (The ‘Break’ Test)

RAID 5 can survive the loss of a single drive. I put this to the test by simulating a hard drive failure:

11. Create a secret file on the RAID:
 `echo "RAID is my secret data!" | sudo tee /mnt/raid_lab/secret.txt`
12. Simulate a disk failure: `sudo mdadm /dev/md0 --fail /dev/loop1`
13. Verify the data still exists: `cat /mnt/raid_lab/secret.txt`
14. Check the 'injured' status: `cat /proc/mdstat`

Part 4: The Repair (Hot-Swap)

To restore the server to full health, I simulated a hot-swap by clearing out the failed "hardware" and re-introducing it as a fresh drive:

15. Remove the broken disk: `sudo mdadm /dev/md0 --remove /dev/loop1`
16. Wipe the disk: `sudo wipefs -a /dev/loop1`
17. Add the new disk back to the array: `sudo mdadm /dev/md0 --add /dev/loop1`
18. Final health check: `sudo mdadm --detail /dev/md0`

Lessons Learned

- **Adaptability:** When local hardware restrictions stopped the project, cloud infrastructure provided an immediate, secure solution.
- **Storage Resilience:** Experienced firsthand how RAID 5 calculates parity mathematically to keep data alive and readable even during an active hardware failure.
- **System Administration:** Mastered core sysadmin tools, including `mdadm`, `losetup`, and filesystem mounting procedures.